

Visualization Literacy Analysis

Derek Harter, Shulan Lu

10/19/2025

Read in data files

```
#grab files from google drive (only have to do this once)
# source("getFromGoogleDrive.R")

#get the first 6? characters of each data file
#get unique values of these
# this is list of subject ids

raw_file_names <- list.files("../AccData")
first_six <- substr(raw_file_names, 1, 6)
sub_ids <- unique(first_six)

# fast_RT = data.frame(ParticipantId = character(),
#                      TrialName = character(),
#                      type = character(),
#                      time = numeric()
# )

all_data <- NULL

for (i in 1:length(sub_ids)){

  if (grepl("~$", sub_ids[i], fixed = T)){
    next
  }

  temp_file1 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_1.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  temp_file2 <- read_xlsx(paste0("../AccData/", sub_ids[i], "_2.xlsx")) %>%
    slice(1:17) %>%
    select(-starts_with("Order")) %>%
    rename(correct = 8)

  new_temp <- temp_file1 %>%
    bind_cols(temp_file2$correct) %>%
    rename(Correct_1 = correct, Correct_2 = "...9") %>%
    mutate(AnswerRT = TimeToBeginInput - TimeToReadQuestion)
```

```

all_data <- all_data %>%
  bind_rows(new_temp)
}

all_data <- all_data %>%
  rename(readRT = TimeToReadQuestion, totalRT = TimeToBeginInput)

#read in trialtype key (I created this from an early version of the previous paper)
trial_type_key <- read.csv("../trial_type_key.csv", stringsAsFactors = F)

all_data <- all_data %>%
  mutate(TrialType = trial_type_key$TrialType[match(TrialName, trial_type_key$TrialName)]) %>%
  mutate(TrialType = paste0("Type", TrialType))

```

Basic checks

```

#how many participants per condition
all_data %>%
  group_by(ParticipantId, Condition) %>%
  summarize(ntrials = n()) %>%
  group_by(Condition) %>%
  summarize(nsubs = n()) %>%
  kable()

```

`summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups` argument.

Condition	nsubs
VR	50
VR Monitor	39
VR Monitor Stereo	33

#why is the balance so off?

Remove outliers based on RT

```

#removing on trial-by-trial basis

#remove answerRTs below 2000ms first
all_data_remove <- all_data %>%
  filter(AnswerRT >= 2000)
dim(all_data)[1]

```

```
## [1] 2074
```

```
dim(all_data_remove)[1]
```

```
## [1] 2067
```

```
#drops 7 trials
```

```
rt_data_summary <- all_data %>%  
  group_by(TrialName) %>%  
  summarize(meanAnswerRT = mean(AnswerRT, na.rm = T),  
            sdAnswerRT = sd(AnswerRT, na.rm = T),  
            UB = meanAnswerRT + 3*sdAnswerRT,  
            LB = meanAnswerRT - 3*sdAnswerRT)  
rt_data_summary
```

```
## # A tibble: 17 x 5  
##   TrialName      meanAnswerRT sdAnswerRT      UB      LB  
##   <chr>          <dbl>      <dbl>    <dbl>  <dbl>  
## 1 BarChartQ1      27611.    14762.   71897. -16675.  
## 2 BarChartQ2      16921.    13338.   56935. -23093.  
## 3 BarChartQ3      15609.     9948.  45452. -14235.  
## 4 BarChartQ4      10288.     8978.  37222. -16646.  
## 5 LineChartQ1      49185.   29505. 137699. -39329.  
## 6 LineChartQ2      40127.   31660. 135107. -54854.  
## 7 LineChartQ3      27190.   17504.  79701. -25322.  
## 8 LineChartQ4      14779.   16568.  64483. -34925.  
## 9 LineChartQ5      27877.   17250.  79628. -23874.  
## 10 ScatterplotQ1    32801.   24294. 105682. -40080.  
## 11 ScatterplotQ2    29636.   23980. 101576. -42304.  
## 12 ScatterplotQ3    45580.   33014. 144623. -53463.  
## 13 ScatterplotQ4    35987.   25402. 112194. -40219.  
## 14 ScatterplotQ5    59805.   42684. 187859. -68248.  
## 15 SurfacePlotQ1    56608.   36042. 164733. -51516.  
## 16 SurfacePlotQ2    65110.   50062. 215297. -85076.  
## 17 SurfacePlotQ3    48373.   37424. 160646. -63900.
```

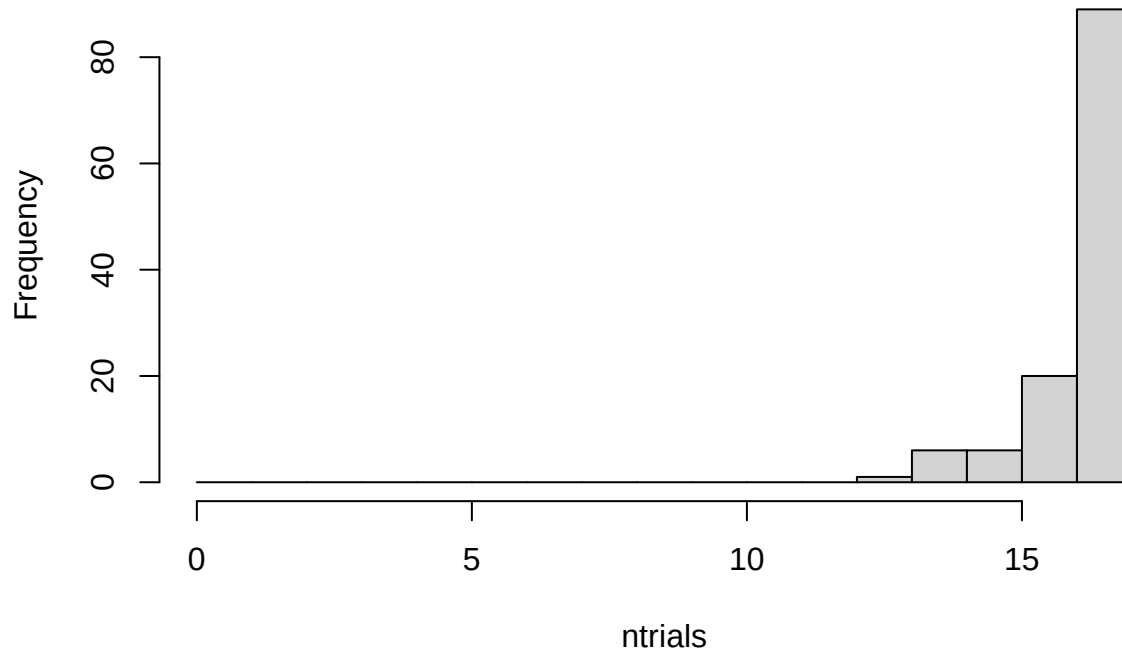
```
all_data_no_outliers <- all_data_remove %>%  
  group_by(TrialName) %>%  
  filter(!(abs(AnswerRT - mean(AnswerRT)) > 3.0*sd(AnswerRT)))  
dim(all_data_no_outliers)[1]
```

```
## [1] 2020
```

```
#drops 47 more trials
```

```
all_data_no_outliers %>%  
  group_by(ParticipantId) %>%  
  summarize(ntrials = n()) %>%  
  with(hist(ntrials, breaks = 0:17))
```

Histogram of ntrials



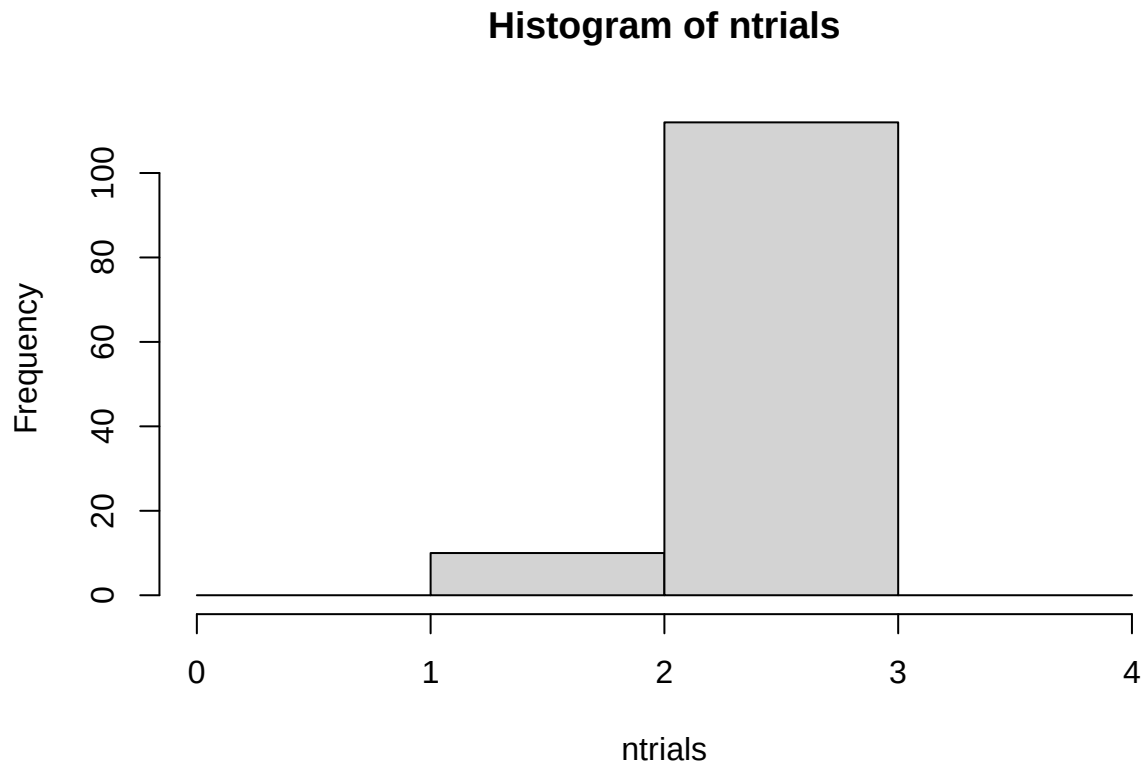
##maybe consider replacing outliers with means instead of removing them?

Extract Only One Type of Chart

Create a dataframe from the data with no outliers that contains only XChartQ1 through XChartQ4 trials for the following analysis.

```
all_data_no_outliers <- all_data_no_outliers %>%  
  filter(grepl("SurfacePlotQ[0-9]", TrialName))
```

```
all_data_no_outliers %>%  
  group_by(ParticipantId) %>%  
  summarize(ntrials = n()) %>%  
  with(hist(ntrials, breaks = 0:4))
```



Compare RTs for conditions and question type (three types: identify, relate, predict)

```
#compare read times (should be no differences of condition)
#compare answer times (potentially a difference)
```

```
trial_type_means <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_readRT = mean(readRT),
             mean_answerRT = mean(AnswerRT),
             n = n())
```

```
## `summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
## the `.groups` argument.
```

```
# first, make .csv files in wide format to double check in statview
```

```
read_rt_type_wider <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_readRT) %>%
  pivot_wider(names_from = TrialType, values_from = mean_readRT)
```

```
# fill in na values with mean for type1, type2 and type3 RT
```

```
# the following is supposed to work, but we get NaN from calculation of mean here, not sure why...
```

```
#read_rt_type_wider_clean <- read_rt_type_wider %>% mutate(across(Type1, ~replace_na(., mean(., na.rm=T))
```

```
read_rt_type_wider_clean <- read_rt_type_wider %>% replace_na(list(Type1 = mean(read_rt_type_wider$Type1, na.rm=T))
```

```
read_rt_type_wider_clean <- read_rt_type_wider_clean %>% replace_na(list(Type2 = mean(read_rt_type_wider$Type2, na.rm=T))
```

```

#read_rt_type_wider_clean <- read_rt_type_wider_clean %>% replace_na(list(Type3 = mean(read_rt_type_wider_clean$Type3)))

answer_rt_type_wider <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_answerRT) %>%
  pivot_wider(names_from = TrialType, values_from = mean_answerRT)
write.csv(read_rt_type_wider, file = "readtypeRTs.csv", row.names = F)
write.csv(answer_rt_type_wider, file = "answertypeRTs.csv", row.names = F)

# fill in na values with mean for type1, type2 and type3 RT
answer_rt_type_wider_clean <- answer_rt_type_wider %>% replace_na(list(Type1 = mean(answer_rt_type_wider$Type1),
  answer_rt_type_wider_clean <- answer_rt_type_wider_clean %>% replace_na(list(Type2 = mean(answer_rt_type_wider$Type2),
#answer_rt_type_wider_clean <- answer_rt_type_wider_clean %>% replace_na(list(Type3 = mean(answer_rt_type_wider$Type3)))

#### READ RTs ####

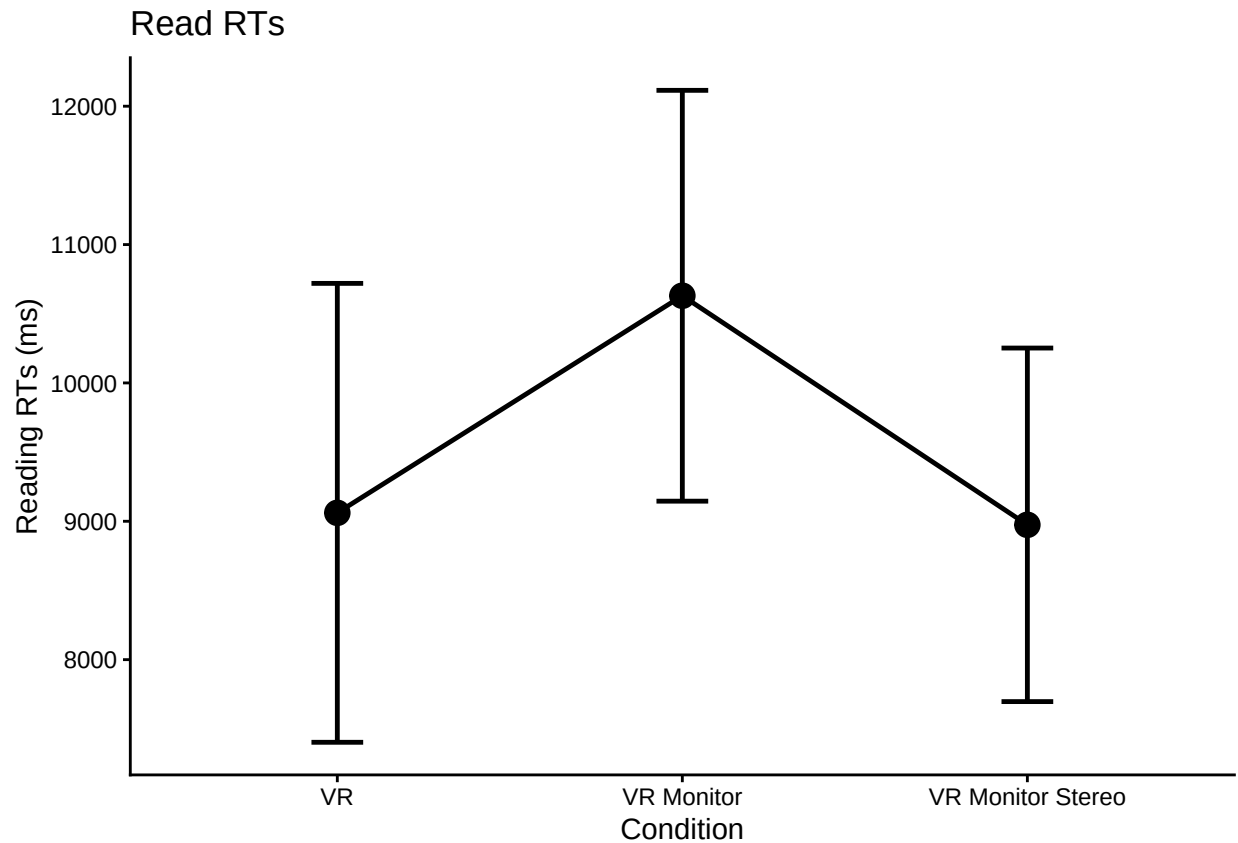
#make some plots

#just condition main effect
readplot1 <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition) %>%
  summarize(overallmean = mean(readRT)) %>%
  group_by(Condition) %>%
  summarize(overall_condition_mean = mean(overallmean),
            se = std.error(overallmean),
            n = n(),
            CI = qt(0.975,df=n-1)*se)

## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.

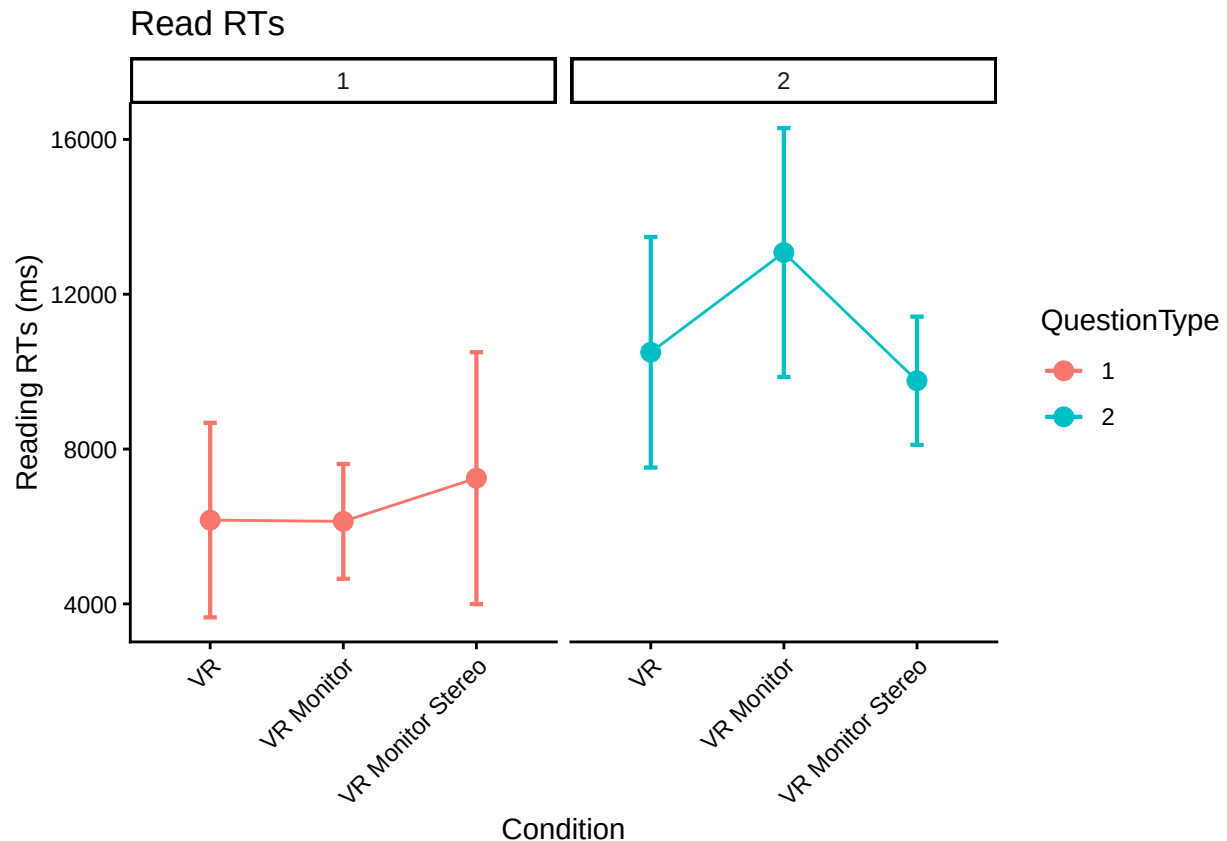
ggplot(readplot1, aes(Condition,
                      overall_condition_mean,
                      group = 1,
                      ymin = overall_condition_mean - CI,
                      ymax = overall_condition_mean + CI)) +
  theme_classic() +
  geom_point(size = 4) +
  geom_errorbar(width = .15, linewidth = 0.85) +
  geom_line(linewidth = 0.85) +
  labs(y = "Reading RTs (ms)", title = "Read RTs")

```



```
#make a little plot using wide format
superbPlot(read_rt_type_wider_clean,
  BSFactors = "Condition",
  WSFactors = "QuestionType(2)",
  variables = c("Type1", "Type2"),
  statistic = "meanNArm",
  errorbar = "CI",
  gamma = 0.95,
  adjustments = list(
    purpose = "difference"
  ),
  plotLayout = "line",
  factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
facet_wrap(vars(QuestionType)) +
labs(y = "Reading RTs (ms)", title = "Read RTs")
```

superb::ADVICE: Some of the groups' variances are heterogeneous. Consider using purpose="tryon".



```
# there are missing values in anova for some participants because they don't
# do all trial types necessarily. Probably a better way to do this, but
# we widen table to make na easier to identify, fill in na with mean of column,
# then make back to narrow version for the anova here
x <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_readRT) %>%
  pivot_wider(names_from = TrialType, values_from = mean_readRT)
x_clean <- x
x_clean$Type1[is.na(x$Type1)] <- mean(x$Type1, na.rm=TRUE)
x_clean$Type2[is.na(x$Type2)] <- mean(x$Type2, na.rm=TRUE)
readRT_means_clean <- x_clean %>%
  pivot_longer(Type1:Type2, names_to="TrialType", values_to = "mean_readRT")

read_rt_type_anova <- aov_ez(id = "ParticipantId",
  dv = "mean_readRT",
  data = readRT_means_clean,
  within = "TrialType",
  between = "Condition",
  anova_table = list(es = "pes") #might want to double-check these
)

## Converting to factor: Condition
## Contrasts set to contr.sum for the following variables: Condition
kable(nice(read_rt_type_anova))
```


Effect	df	MSE	F	pes	p.value
Condition	2, 119	42258798.81	0.92	.015	.400
TrialType	1, 119	29790210.88	42.08 ***	.261	<.001
Condition:TrialType	2, 119	29790210.88	3.03 +	.048	.052

```
#posthoc test for trial type
```

```
pairs(emmeans(read_rt_type_anova, "TrialType"), adjust = "Tukey")
```

```
## contrast      estimate SE df t.ratio p.value
```

```
## Type1 - Type2    -4600 709 119  -6.487  <.0001
```

```
##
```

```
## Results are averaged over the levels of: Condition
```

```
#### ANSWER RTs ####
```

```
#make some plots
```

```
#just condition main effect
```

```
answerplot1 <- all_data_no_outliers %>%
```

```
  group_by(ParticipantId, Condition) %>%
```

```
  summarize(overallmean = mean(AnswerRT)) %>%
```

```
  group_by(Condition) %>%
```

```
  summarize(overall_condition_mean = mean(overallmean),
```

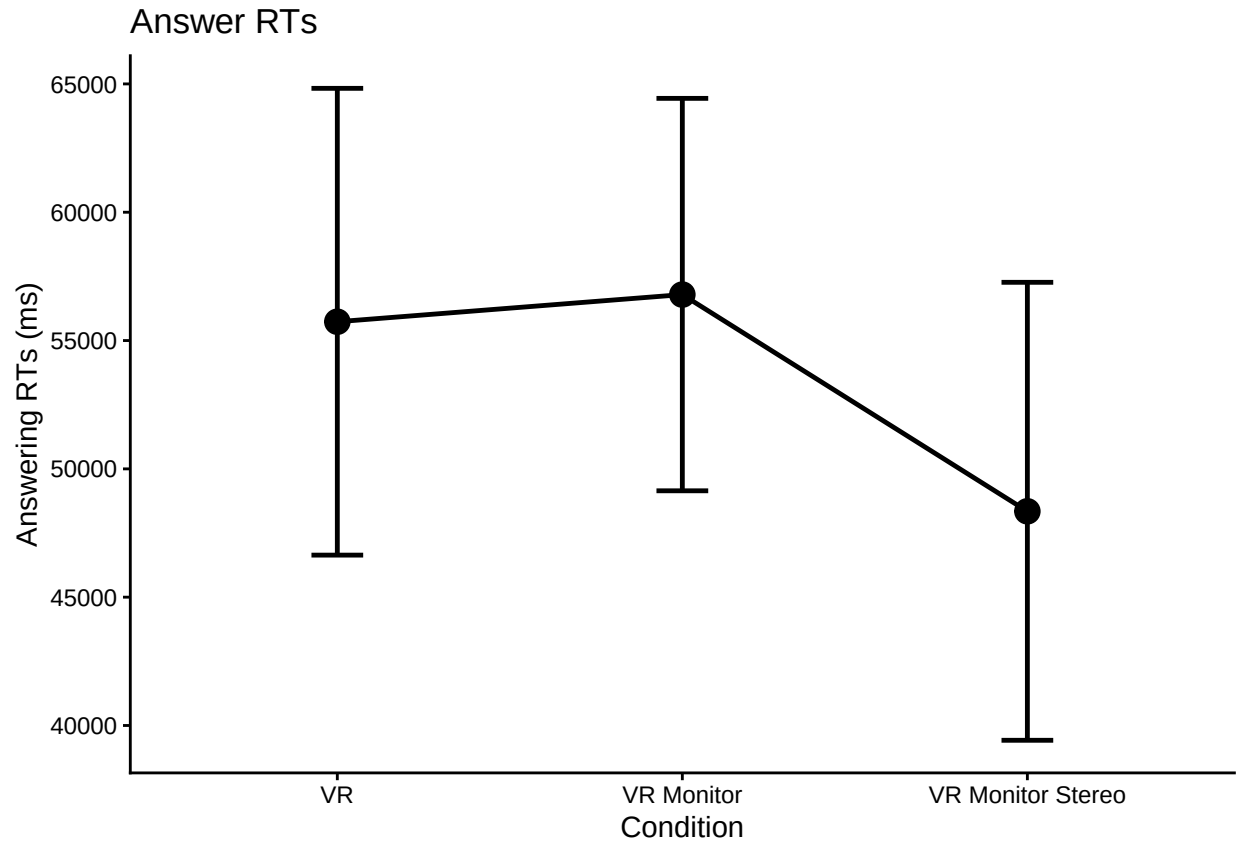
```
            se = std.error(overallmean),
```

```
            n = n(),
```

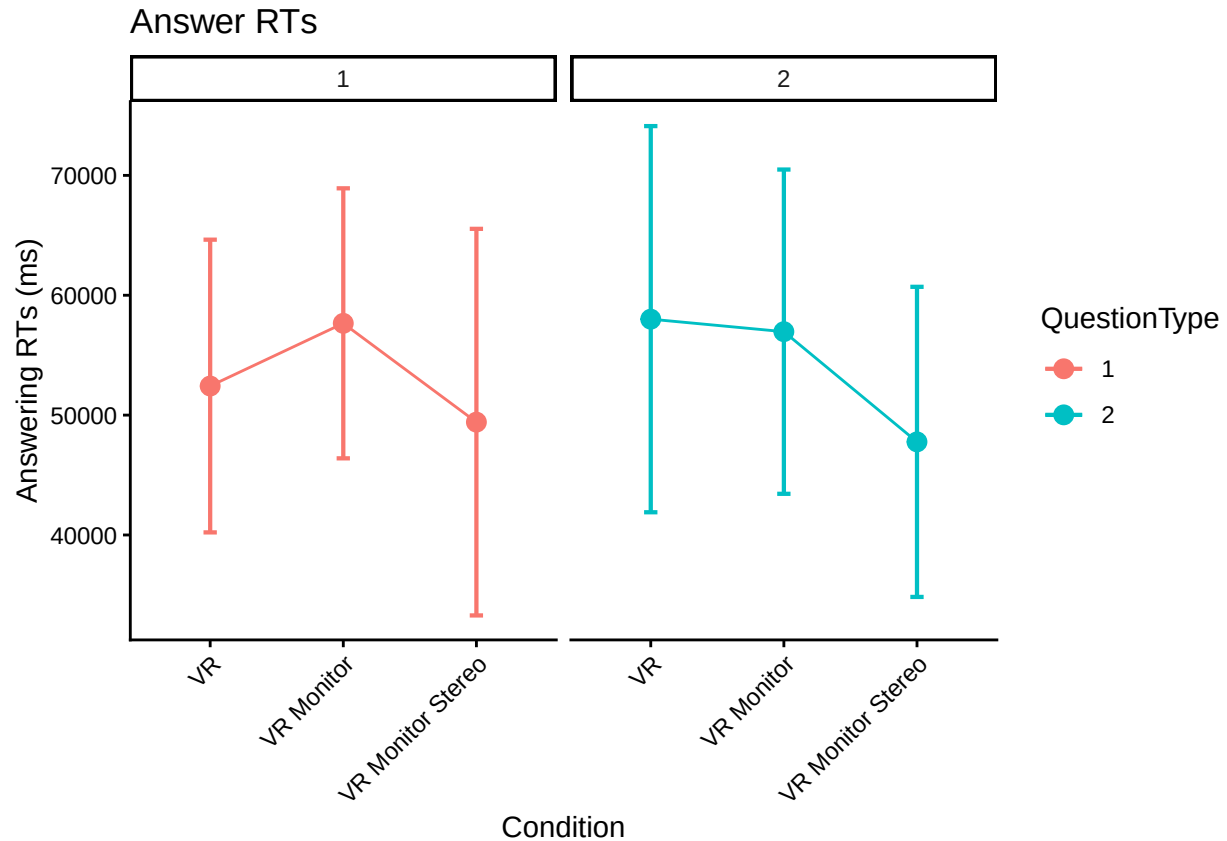
```
            CI = qt(0.975,df=n-1)*se)
```

```
## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`  
## argument.
```

```
ggplot(answerplot1, aes(Condition,  
                        overall_condition_mean,  
                        group = 1,  
                        ymin = overall_condition_mean - CI,  
                        ymax = overall_condition_mean + CI)) +  
  
  theme_classic() +  
  geom_point(size = 4) +  
  geom_errorbar(width = .15, linewidth = 0.85) +  
  geom_line(linewidth = 0.85) +  
  labs(y = "Answering RTs (ms)", title = "Answer RTs")
```



```
#make the little plot
superbPlot(answer_rt_type_wider_clean,
  BSFactors = "Condition",
  WSFactors = "QuestionType(2)",
  variables = c("Type1", "Type2"),
  statistic = "meanNArm",
  errorbar = "CI",
  gamma = 0.95,
  adjustments = list(
    purpose = "difference"
  ),
  plotLayout = "line",
  factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1)) +
facet_wrap(vars(QuestionType)) +
labs(y = "Answering RTs (ms)", title = "Answer RTs")
```



```
# there are missing values in anova for some participants because they don't
# do all trial types necessarily. Probably a better way to do this, but
# we widen table to make na easier to identify, fill in na with mean of column,
# then make back to narrow version for the anova here
x <- trial_type_means %>%
  select(ParticipantId, Condition, TrialType, mean_answerRT) %>%
  pivot_wider(names_from = TrialType, values_from = mean_answerRT)
x_clean <- x
x_clean$Type1[is.na(x$Type1)] <- mean(x$Type1, na.rm=TRUE)
x_clean$Type2[is.na(x$Type2)] <- mean(x$Type2, na.rm=TRUE)
#x_clean$Type3[is.na(x$Type3)] <- mean(x$Type3, na.rm=TRUE)
answerRT_means_clean <- x_clean %>%
  pivot_longer(Type1:Type2, names_to="TrialType", values_to = "mean_answerRT")

answer_rt_type_anova <- aov_ez(id = "ParticipantId",
  dv = "mean_answerRT",
  data = answerRT_means_clean,
  within = "TrialType",
  between = "Condition",
  anova_table = list(es = "pes") #might want to double-check these
)

## Converting to factor: Condition
## Contrasts set to contr.sum for the following variables: Condition
kable(nice(answer_rt_type_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 119	1461316978.43	1.00	.017	.370
TrialType	1, 119	507806748.33	0.14	.001	.713
Condition:TrialType	2, 119	507806748.33	0.66	.011	.518

#posthoc test for trial type

```
pairs(emmeans(answer_rt_type_anova, "TrialType"), adjust = "Tukey")
```

```
## contrast      estimate    SE df t.ratio p.value
## Type1 - Type2    -1079 2930 119  -0.368  0.7132
##
## Results are averaged over the levels of: Condition
```

#Question Type 3 much faster than Type 1/Type 2 (this is answering times)

```
ref <- emmeans(answer_rt_type_anova, ~Condition|TrialType)
```

```
pairs(ref, adjust = "Tukey")
```

```
## TrialType = Type1:
## contrast              estimate    SE df t.ratio p.value
## VR - VR Monitor        -5231 6230 119  -0.839  0.6794
## VR - VR Monitor Stereo   3013 6540 119   0.461  0.8898
## VR Monitor - VR Monitor Stereo  8244 6900 119   1.195  0.4586
##
## TrialType = Type2:
## contrast              estimate    SE df t.ratio p.value
## VR - VR Monitor         1043 7140 119   0.146  0.9883
## VR - VR Monitor Stereo  10236 7500 119   1.365  0.3627
## VR Monitor - VR Monitor Stereo  9193 7910 119   1.162  0.4780
##
## P value adjustment: tukey method for comparing a family of 3 estimates
```

#plot and interaction suggests that the conditions have different effects for different question types. posthoc tests indicate a difference between VR and VRMonitor for QType 1

Accuracy

```
# do interrater reliability: Fits each trial as independent with two raters
# That is, this ignores participants (complete pooling, which may be problematic)
# 1410 trials from N = 85 participants, not exactly 17 trials per participant
# b/c of excluded trials
irr::icc(select(ungroup(all_data_no_outliers), Correct_1, Correct_2))
```

```
## Single Score Intraclass Correlation
##
## Model: oneway
## Type : consistency
##
## Subjects = 356
## Raters = 2
## ICC(1) = 0.772
##
## F-Test, H0: r0 = 0 ; H1: r0 > 0
```

```
## F(355,356) = 7.76 , p = 3.21e-72
##
## 95%-Confidence Interval for ICC Population Values:
## 0.726 < ICC < 0.811

head(all_data_no_outliers)

## # A tibble: 6 x 11
## # Groups:   TrialName [3]
##   ParticipantId Condition TrialNumber TrialName readRT totalRT UserInput Correct_1
##   <chr>          <chr>      <dbl> <chr>      <dbl>    <dbl> <chr>      <dbl>
## 1 AA0176        VR Monitor      15 SurfacePlotQ1 5834. 77172. When crime ~ 0.5
## 2 AA0176        VR Monitor      16 SurfacePlotQ2 13170. 108782. The crime i~ 1
## 3 AA0176        VR Monitor      17 SurfacePlotQ3 21458. 62138. When crime ~ 1
## 4 AA0876        VR              6 SurfacePlotQ1 8343. 38400. 90.0 0
## 5 AA0876        VR              7 SurfacePlotQ2 10166. 48544. when high p~ 1
## 6 AA0876        VR              8 SurfacePlotQ3 7361. 19481. percentage ~ 0
## # i 3 more variables: Correct_2 <dbl>, AnswerRT <dbl>, TrialType <chr>

# account for non-independence?
#Data has to be narrow
#https://cran.r-project.org/web/packages/cccrm/index.html
icc.dat.narrow <- all_data_no_outliers %>%
  pivot_longer(cols = c('Correct_1', 'Correct_2'))

colnames(icc.dat.narrow)[10] <- "Rater"
icc.dat.narrow$Rater <- as.factor(icc.dat.narrow$Rater)

#Longitudinal concordance for two raters
#By all 17 trials (by items)
# icc.rm.trial.name <- ccclon(icc.dat.narrow,
#                             "value", "ParticipantId", "TrialName", "Rater")
# icc.rm.trial.name
# summary(icc.rm.trial.name)

#By time using trial numbers (by time)
icc.rm.trial.number <- ccclon(icc.dat.narrow,
                             "value", "ParticipantId", "TrialNumber", "Rater" )

## Warning: `ccclon()` was deprecated in cccrm 3.0.0.
## i Please use `ccc_vc()` instead.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was generated.

icc.rm.trial.number

## CCC estimated by variance components:
##      CCC LL CI 95% UL CI 95%      SE CCC
## 0.76129551 0.70859040 0.80555307 0.02465609

summary(icc.rm.trial.number)

##      Subjects Subjects-Method Subjects-Time      Method      Error
##      0.066943475      0.001008501      0.084326956      0.000000000      0.046422399
##
## CCC estimated by variance compoments
##      CCC LL CI 95% UL CI 95%      SE CCC
## 0.76129551 0.70859040 0.80555307 0.02465609
```

```

# A similar approach
#https://peerj.com/articles/9850/
# test.mod <-
#   lcc(data = hue, subject = "Fruit", resp = "H_mean",
#       method = "Method", time = "Time", qf = 2, qr = 1)
# summary(test.mod)

# icc.dat.narrow$ParticipantId <- as.factor(icc.dat.narrow$ParticipantId)
# icc.dat.narrow$Rater <- as.factor(icc.dat.narrow$Rater)
#
#
# icc.rm.another <-
# lcc(data = icc.dat.narrow,
#     subject = "ParticipantId",
#     resp = "value",
#     method = "Rater",
#     time = "TrialNumber",
#     qf = 1, #polynomial trends (1 to 3)
#     qr = 0, #random effects (0 is random int only, default)
#     components = TRUE)
#
# #LCC = Longitudinal Concordance Correlation
# #LPC = Longitudinal Pearson Correlation
# #LA = Longitudinal Accuracy
# icc.rm.another
#
# #Model coefficients near 1 but only the the last one has a correctly
# #estimated model. Maybe because it's more or less flat over TrialNumber?
# lccPlot(icc.rm.another, type = "lpc")
# lccPlot(icc.rm.another, type = "lcc")
# lccPlot(icc.rm.another, type = "la")

all_data_no_outliers <- all_data_no_outliers %>%
  rowwise() %>%
  mutate(Correct_Avg = mean(c(Correct_1, Correct_2)))

#just condition main effect
accplot1 <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition) %>%
  summarize(overallmean = mean(Correct_Avg)) %>%
  group_by(Condition) %>%
  summarize(overall_condition_mean = mean(overallmean),
            se = std.error(overallmean),
            n = n(),
            CI = qt(0.975,df=n-1)*se)

## `summarise()` has grouped output by 'ParticipantId'. You can override using the `.groups`
## argument.

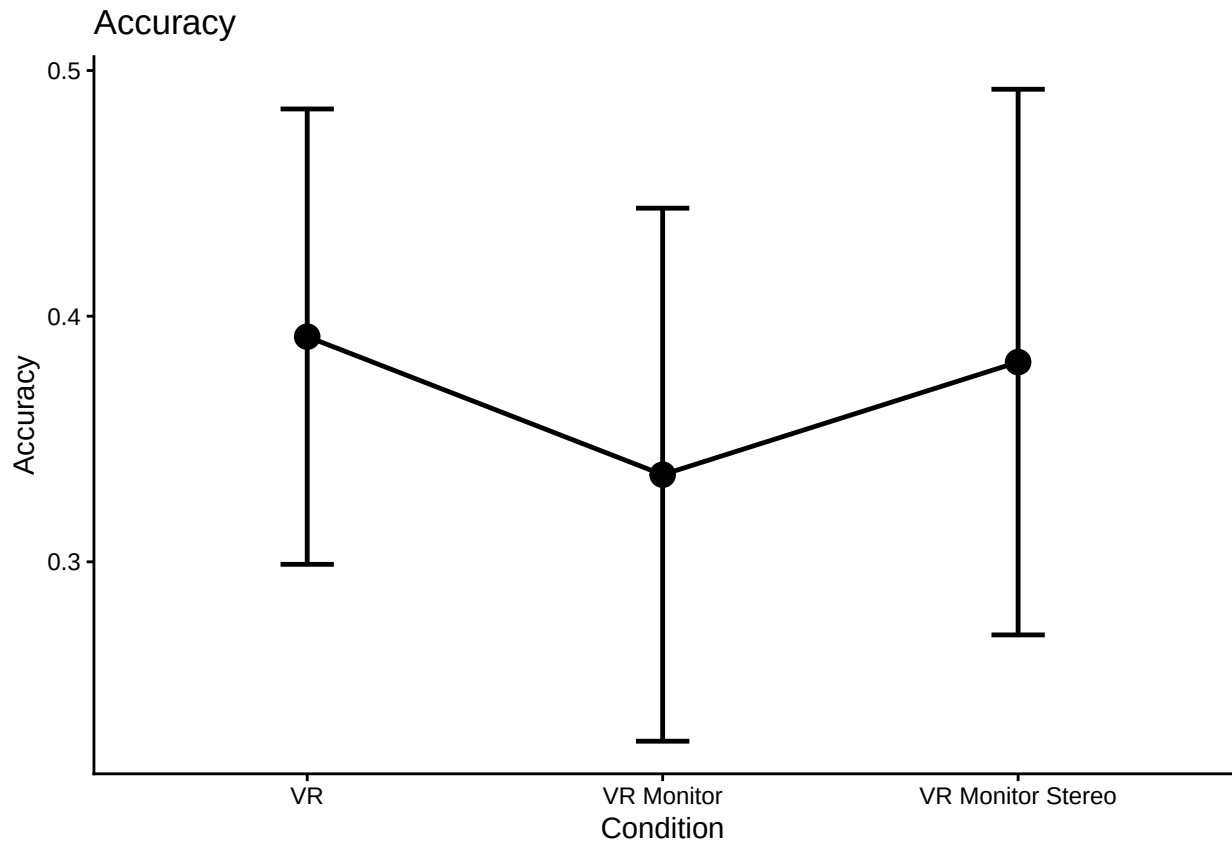
ggplot(accplot1, aes(Condition,
                     overall_condition_mean,
                     group = 1,
                     ymin = overall_condition_mean - CI,

```

```

      ymax = overall_condition_mean + CI)) +
theme_classic() +
geom_point(size = 4) +
geom_errorbar(width = .15, linewidth = 0.85) +
geom_line(linewidth = 0.85) +
labs(y = "Accuracy", title = "Accuracy")

```



```

trial_type_mean_acc <- all_data_no_outliers %>%
  group_by(ParticipantId, Condition, TrialType) %>%
  summarize(mean_acc = mean(Correct_Avg),
            n = n())

```

`summarise()` has grouped output by 'ParticipantId', 'Condition'. You can override using
the `.groups` argument.

```

acc_type_wider <- trial_type_mean_acc %>%
  select(ParticipantId, Condition, TrialType, mean_acc) %>%
  pivot_wider(names_from = TrialType, values_from = mean_acc)
write.csv(acc_type_wider, file = "typeacc.csv", row.names = F)

```

replace na with mean

```

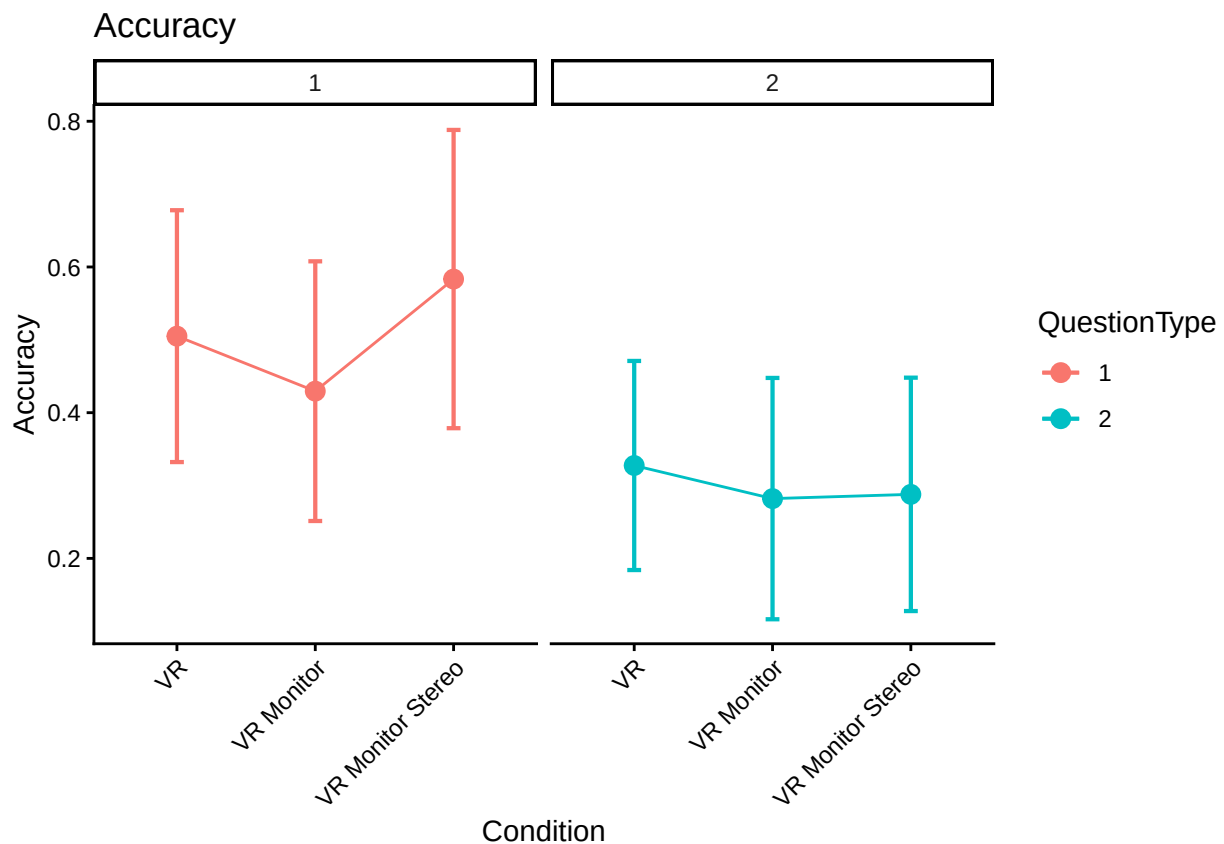
acc_type_wider_clean <- acc_type_wider %>% replace_na(list(Type1 = mean(acc_type_wider$Type1, na.rm=TRUE)
acc_type_wider_clean <- acc_type_wider_clean %>% replace_na(list(Type2 = mean(acc_type_wider$Type2, na.rm=TRUE)
acc_type_wider_clean <- acc_type_wider_clean %>% replace_na(list(Type3 = mean(acc_type_wider$Type3, na.rm=TRUE)

```

Warning: Unknown or uninitialised column: `Type3`.

Warning in mean.default(acc_type_wider\$Type3, na.rm = TRUE): argument is not numeric or

```
## logical: returning NA
#make the little plot
superbPlot(acc_type_wider_clean,
  BSFactors = "Condition",
  WSFactors = "QuestionType(2)",
  variables = c("Type1", "Type2"),
  statistic = "meanNArm",
  errorbar = "CI",
  gamma = 0.95,
  adjustments = list(
    purpose = "difference"
  ),
  plotLayout = "line",
  factorOrder = c("Condition", "QuestionType")
) +
theme_classic() +
theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust=1))+
facet_wrap(vars(QuestionType))+
labs(y = "Accuracy", title = "Accuracy")
```



```
# there are missing values in anova for some participants because they don't
# do all trial types necessarily. Probably a better way to do this, but
# we widen table to make na easier to identify, fill in na with mean of column,
# then make back to narrow version for the anova here
x <- trial_type_mean_acc %>%
  select(ParticipantId, Condition, TrialType, mean_acc) %>%
```



```

    pivot_wider(names_from = TrialType, values_from = mean_acc)
x_clean <- x
x_clean$Type1[is.na(x$Type1)] <- mean(x$Type1, na.rm=TRUE)
x_clean$Type2[is.na(x$Type2)] <- mean(x$Type2, na.rm=TRUE)
#x_clean$Type3[is.na(x$Type3)] <- mean(x$Type3, na.rm=TRUE)
acc_means_clean <- x_clean %>%
  pivot_longer(Type1:Type2, names_to="TrialType", values_to = "mean_acc")

acc_type_anova <- aov_ez(id = "ParticipantId",
  dv = "mean_acc",
  data = acc_means_clean,
  within = "TrialType",
  between = "Condition",
  anova_table = list(es = "pes") #might want to double-check these
)

```

```
## Converting to factor: Condition
```

```
## Contrasts set to contr.sum for the following variables: Condition
```

```
kable(nice(acc_type_anova))
```

Effect	df	MSE	F	pes	p.value
Condition	2, 119	0.22	0.60	.010	.549
TrialType	1, 119	0.07	34.37 ***	.224	<.001
Condition:TrialType	2, 119	0.07	1.47	.024	.234

```
#posthoc test for trial type
```

```
pairs(emmeans(acc_type_anova, "TrialType"), adjust = "Tukey")
```

```
## contrast      estimate      SE  df t.ratio p.value
## Type1 - Type2    0.207 0.0353 119   5.862 <.0001
##
```

```
## Results are averaged over the levels of: Condition
```

```
#Question Type 3 much faster than Type 1/Type 2 (this is answering times)
```

```
ref <- emmeans(acc_type_anova, ~Condition | TrialType)
```

```
pairs(ref, adjust = "Tukey")
```

```
## TrialType = Type1:
```

```
## contrast              estimate      SE  df t.ratio p.value
## VR - VR Monitor        0.07550 0.0879 119   0.859  0.6671
## VR - VR Monitor Stereo -0.07835 0.0923 119  -0.849  0.6732
## VR Monitor - VR Monitor Stereo -0.15386 0.0973 119  -1.581  0.2577
##
```

```
## TrialType = Type2:
```

```
## contrast              estimate      SE  df t.ratio p.value
## VR - VR Monitor        0.04545 0.0745 119   0.610  0.8149
## VR - VR Monitor Stereo  0.03962 0.0782 119   0.507  0.8683
## VR Monitor - VR Monitor Stereo -0.00583 0.0825 119  -0.071  0.9973
##
```

```
## P value adjustment: tukey method for comparing a family of 3 estimates
```